



Kth Smallest Number in M Sorted Lists (Medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
 - Solution
 - Code
 - Time complexity
 - Space complexity
- Similar Problems

Problem Statement

Given 'M' sorted arrays, find the K'th smallest number among all the arrays.

Example 1:

Input: L1=[2, 6, 8], L2=[3, 6, 7], L3=[1, 3, 4], K=5

Output: 4

Explanation: The 5th smallest number among all the arrays is 4, this can be verified from the merged list of all the arrays: [1, 2, 3, 3, 4, 6, 6, 7, 8]

Example 2:

Input: L1=[5, 8, 9], L2=[1, 7], K=3

Output: 7

Explanation: The 3rd smallest number among all the arrays is 7.

Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class KthSmallestInMSortedArrays {
4
5     public static int findKthSmallest(List<Integer[]> lists, int k) {
6         // TODO: Write your code here
7         return -1;
8     }
9
10    public static void main(String[] args) {
```



```

11  integer[] l1 = new Integer[] { 2, 6, 8 };
12  Integer[] l2 = new Integer[] { 3, 6, 7 };
13  Integer[] l3 = new Integer[] { 1, 3, 4 };
14  List<Integer[]> lists = new ArrayList<Integer[]>();
15  lists.add(l1);
16  lists.add(l2);
17  lists.add(l3);
18  int result = KthSmallestInMSortedArrays.findKthSmallest(lists, 5);
19  System.out.println("Kth smallest number is: " + result);
20  }
21  }
22

```

Run

Save

Reset



Solution

This problem follows the **K-way merge** pattern and we can follow a similar approach as discussed in [Merge K Sorted Lists](#).

We can start merging all the arrays, but instead of inserting numbers into a merged list, we will keep count to see how many elements have been inserted in the merged list. Once that count is equal to 'K', we have found our required number.

A big difference from [Merge K Sorted Lists](#) is that in this problem, the input is a list of arrays compared to LinkedLists. This means that when we want to push the next number in the heap we need to know what the index of the current number in the current array was. To handle this, we will need to keep track of the array and the element indices.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class Node {
4      int elementIndex;
5      int arrayIndex;
6
7      Node(int elementIndex, int arrayIndex) {
8          this.elementIndex = elementIndex;
9          this.arrayIndex = arrayIndex;
10     }
11 }
12
13 class KthSmallestInMSortedArrays {
14
15     public static int findKthSmallest(List<Integer[]> lists, int k) {
16         PriorityQueue<Node> minHeap = new PriorityQueue<Node>(
17             (n1, n2) -> lists.get(n1.arrayIndex)[n1.elementIndex] - lists.get(n2.arrayIndex)[n2.elementIndex]);
18
19         // put the 1st element of each array in the min heap
20         for (int i = 0; i < lists.size(); i++)
21             if (lists.get(i) != null)
22                 minHeap.add(new Node(0, i));
23
24         // take the smallest (top) element form the min heap, if the running count is equal to k return the number
25         // if the array of the top element has more elements, add the next element to the heap

```

```

26     int numberCount = 0, result = 0;
27     while (!minHeap.isEmpty()) {
28         Node node = minHeap.poll();
29         result = lists.get(node.arrayIndex)[node.elementIndex];
30         if (++numberCount == k)
31             break;
32         node.elementIndex++;
33         if (lists.get(node.arrayIndex).length > node.elementIndex)
34             minHeap.add(node);
35     }
36     return result;
37 }
38
39 public static void main(String[] args) {
40     Integer[] l1 = new Integer[] { 2, 6, 8 };
41     Integer[] l2 = new Integer[] { 3, 6, 7 };
42     Integer[] l3 = new Integer[] { 1, 3, 4 };
43     List<Integer[]> lists = new ArrayList<Integer[]>();
44     lists.add(l1);
45     lists.add(l2);
46     lists.add(l3);
47     int result = KthSmallestInMSortedArrays.findKthSmallest(lists, 5);
48     System.out.print("Kth smallest number is: " + result);
49 }
50 }
51

```

Run

Save

Reset



Time complexity

Since we'll be going through at most 'K' elements among all the arrays, and we will remove/add one element in the heap in each step, the time complexity of the above algorithm will be $O(K * \log M)$ where 'M' is the total number of input arrays.

Space complexity

The space complexity will be $O(M)$ because, at any time, our min-heap will be storing one number from all the 'M' input arrays.

Similar Problems

Problem 1: Given 'M' sorted arrays, find the median number among all arrays.

Solution: This problem is similar to our parent problem with $K = \text{Median}$. So if there are 'N' total numbers in all the arrays we need to find the K'th minimum number where $K = N/2$.

Problem 2: Given a list of 'K' sorted arrays, merge them into one sorted list.

Solution: This problem is similar to [Merge K Sorted Lists](#) except that the input is a list of arrays compared to **LinkedLists**. To handle this, we can use a similar approach as discussed in our parent problem by keeping a track of the array and the element indices.


[← Back](#)
☒ [Mark As Completed](#)
[Next](#) -

